Cracking Pandas DateTime

From Seconds to Years — A Complete Hands-On Guide

Kuntal Chakraborty

Experience in — Data Science | AI/ML Engineer | Business Analyst

A practical reference covering every date/time granularity,
strftime codes, arithmetic, resampling, timezones, business days and more.

Table of Contents

Table of Contents 2
Chapter 1 — Introduction to Dates & Times in Pandas 4
1.1 Why Date Handling Matters 4
1.2 Core Date/Time Types in Pandas 4
1.3 Quick Setup 4
Chapter 2 — Creating Date & Time Objects 6
2.1 pd.Timestamp() 6
2.2 pd.to_datetime() 6
2.3 pd.date_range() 7
2.4 pd.period_range() and pd.timedelta_range() 8
Chapter 3 — strftime & strptime: Formatting and Parsing Dates 9
3.1 Complete strftime Format Code Reference 9
3.2 strftime on a Single Timestamp 10
3.3 strftime on a DataFrame Column 10
3.4 strptime — Parsing Strings Back to Dates 11
3.5 Regional Date Formats 12
Chapter 4 — The .dt Accessor: Extracting Every Granularity 14
4.1 Setup DataFrame for This Chapter 14
4.2 Year 14
4.3 Half-Year 14
4.4 Quarter 15
4.5 Month 15
4.6 Day 15
4.7 Hour 16
4.8 Minute 16
4.9 Second 16
Chapter 5 — Date Arithmetic: Addition & Subtraction at Every Granularity 18
5.1 Key Arithmetic Tools 18
5.2 Year-Level Arithmetic 18
5.3 Half-Year Arithmetic 19
5.4 Quarter Arithmetic 19
5.5 Month Arithmetic 19
5.6 Day Arithmetic 20
5.7 Hour Arithmetic 21
5.8 Minute Arithmetic 21

5.9 Second Arithmetic 22
5.10 Timedelta Properties and Decomposition 23
Chapter 6 — Resampling & Frequency Conversion 24
6.1 Offset Alias Quick Reference 24
6.2 Downsampling — Daily to Monthly 24
6.3 Downsampling at Every Granularity 25
6.4 Upsampling with Fill Methods 26
Chapter 7 — Time Zones in Pandas 28
7.1 tz_localize vs tz_convert 28
7.2 Timezone Operations on a Series / DataFrame 28
7.3 DST Pitfalls & Ambiguous Times 29
Chapter 8 — Filtering & Selecting by Date 31
8.1 Boolean Indexing with Date Comparisons 31
8.2 .loc[] with Date Strings — DatetimeIndex 31
8.3 between_time() and at_time() 32
Chapter 9 — Business Days & Custom Calendars 34
9.1 Business Day Offsets 34
9.2 Counting Business Days with np.busday_count() 34
9.3 Custom Holiday Calendars 35
Chapter 10 — Practical Real-World Use Cases 37
10.1 Sales Time Series Analysis 37
10.2 Finding & Filling Missing Dates 37
10.3 Age Calculator with Full Granularity 38
10.4 Session Duration Analysis 39
Appendix — Quick Reference Cheat Sheet 41
A.1 Creation 41
A.2 .dt Accessor 41
A.3 Arithmetic 41
A.4 Resampling 41
A.5 Filtering 41

Chapter 1 — Introduction to Dates & Times in Pandas

Date and time handling is one of the most critical skills in data science and analytics. Whether you are analysing sales trends, monitoring server logs, forecasting stock prices, or studying patient records, almost every real-world dataset contains a time dimension. Python's pandas library provides a rich, high-performance toolkit for working with dates and times at every level of granularity — from years down to individual seconds.

1.1 Why Date Handling Matters

Raw data often stores dates as plain strings such as '2024-07-15' or integers like 20240715. Before you can slice, aggregate, or compute differences, these must be converted to proper datetime objects. Pandas makes this conversion seamless and then exposes hundreds of vectorised operations through a single .dt accessor.

1.2 Core Date/Time Types in Pandas

pandas.Timestamp — Represents a single point in time. It is pandas' equivalent of Python's datetime.datetime and is the fundamental building block.

pandas.DatetimeIndex — An index of Timestamp values. Enables fast label-based date slicing.

pandas.Period — Represents a span of time (e.g., a whole month or quarter) rather than an instant.

pandas.Timedelta — Represents a duration or difference between two timestamps.

pandas.NaT — 'Not a Time' — the date equivalent of NaN, used for missing timestamps.

1.3 Quick Setup

Import Conventions Used Throughout This Book

Note: All code examples in this book assume pandas >= 1.3 and Python >= 3.8. Install with: pip install pandas python-dateutil

Exercises

Exercise 1 [Easy]

Import pandas and print its version using pd.__version__

Exercise 2 [Easy]

Create a pandas Timestamp for today's date using pd.Timestamp.now()

Exercise 3 [Medium]

Create a Series of 5 date strings and convert them to datetime using pd.to_datetime()

Exercise 4 [Medium]

Create a DatetimeIndex for the first 7 days of January 2024

Exercise 5 [Hard]

Create a DataFrame with 100 rows and a 'date' column starting 2020-01-01 incrementing daily. Then filter rows where the date falls on a Monday.

Exercise 6 [Hard]

Given a mixed-format date Series ['2024-01-15', '15/02/2024', 'March 3 2024'], convert all to datetime correctly using pd.to_datetime with errors='coerce' and identify any NaT values.

Chapter 2 — Creating Date & Time Objects

Before working with dates you need to create them. This chapter covers every method for creating individual timestamps, date ranges, period ranges, and timedelta ranges.

2.1 pd.Timestamp()

pd.Timestamp is the single-value date/time object in pandas. You can construct it from strings, keyword arguments, or existing Python datetime objects.

Creating Timestamps

```
# From a string
ts1 = pd.Timestamp('2024-07-15 14:30:45')
print(ts1)

# From keyword arguments
ts2 = pd.Timestamp(year=2024, month=7, day=15, hour=14, minute=30, second=45)
print(ts2)

# From a Python datetime
from datetime import datetime
ts3 = pd.Timestamp(datetime(2024, 7, 15, 14, 30, 45))
print(ts3)

# Current time
ts4 = pd.Timestamp.now()
print(ts4)
```

Output:

```
2024-07-15 14:30:45
2024-07-15 14:30:45
2024-07-15 14:30:45
2024-07-15 14:30:45.123456   # (actual current time)
```

2.2 pd.to_datetime()

This is the most-used conversion function. It handles single values, lists, Series, and entire DataFrame columns.

Converting Various Inputs

```
# Single string
print(pd.to_datetime('2024-07-15'))

# List of strings
dates = pd.to_datetime(['2024-01-01', '2024-06-15', '2024-12-31'])
print(dates)

# DataFrame column
df = pd.DataFrame({'date_str': ['2024-01-01', '2024-06-15', '2024-12-31'],
                   'value': [100, 200, 300]})
df['date'] = pd.to_datetime(df['date_str'])
print(df.dtypes)

# With custom format
pd.to_datetime('15-07-2024', format='%d-%m-%Y')
```

```
# Unix timestamps (seconds since epoch)
pd.to_datetime(1720000000, unit='s')

# Handling errors
pd.to_datetime(['2024-01-01', 'not_a_date', '2024-12-31'], errors='coerce')
```

Output:

```
2024-07-15 00:00:00
DatetimeIndex(['2024-01-01', '2024-06-15', '2024-12-31'], dtype='datetime64[ns]')
date_str     object
value         int64
date     datetime64[ns]
Timestamp('2024-07-15 00:00:00')
Timestamp('2024-07-03 21:46:40')
DatetimeIndex(['2024-01-01', 'NaT', '2024-12-31'], dtype='datetime64[ns]')
```

2.3 pd.date_range()

Generates a fixed-frequency DatetimeIndex. This is essential for creating time series data, filling missing date gaps, and building calendar scaffolds.

date_range with Various Frequencies

```
# Daily range
dr_daily = pd.date_range(start='2024-01-01', end='2024-01-07', freq='D')
print(dr_daily)

# Using periods instead of end
dr_periods = pd.date_range(start='2024-01-01', periods=6, freq='MS')  # Month Start
print(dr_periods)

# Hourly range
dr_hourly = pd.date_range(start='2024-01-01', periods=24, freq='H')
print(dr_hourly[:5])

# Business days only
dr_biz = pd.date_range(start='2024-01-01', periods=5, freq='B')
print(dr_biz)

# Minute-level
dr_min = pd.date_range(start='2024-01-01 09:00', periods=5, freq='T')
print(dr_min)

# Second-level
dr_sec = pd.date_range(start='2024-01-01 09:00:00', periods=5, freq='S')
print(dr_sec)
```

Output:

```
DatetimeIndex(['2024-01-01','2024-01-02','2024-01-03','2024-01-04',
               '2024-01-05','2024-01-06','2024-01-07'], freq='D')
DatetimeIndex(['2024-01-01','2024-02-01','2024-03-01','2024-04-01',
               '2024-05-01','2024-06-01'], freq='MS')
DatetimeIndex(['2024-01-01 00:00:00','2024-01-01 01:00:00','2024-01-01 02:00:00',
               '2024-01-01 03:00:00','2024-01-01 04:00:00'], freq='H')
DatetimeIndex(['2024-01-01','2024-01-02','2024-01-03','2024-01-04','2024-01-05'],
freq='B')
DatetimeIndex(['2024-01-01 09:00','2024-01-01 09:01','2024-01-01 09:02',
```

```
                '2024-01-01 09:03','2024-01-01 09:04'], freq='T')
DatetimeIndex(['2024-01-01 09:00:00','2024-01-01 09:00:01','2024-01-01 09:00:02',
                '2024-01-01 09:00:03','2024-01-01 09:00:04'], freq='S')
```

2.4 pd.period_range() and pd.timedelta_range()

```
# Period range — months
pr = pd.period_range(start='2024-01', periods=6, freq='M')
print(pr)

# Period range — quarters
pr_q = pd.period_range(start='2024Q1', periods=4, freq='Q')
print(pr_q)

# Timedelta range
td = pd.timedelta_range(start='1 day', periods=5, freq='6H')
print(td)
```

Output:

```
PeriodIndex(['2024-01','2024-02','2024-03','2024-04','2024-05','2024-06'],
dtype='period[M]')
PeriodIndex(['2024Q1','2024Q2','2024Q3','2024Q4'], dtype='period[Q-DEC]')
TimedeltaIndex(['1 days 00:00:00','1 days 06:00:00','1 days 12:00:00',
                '1 days 18:00:00','2 days 00:00:00'], freq='6H')
```

Exercises

Exercise 1 [Easy]

Use pd.date_range to generate every day in February 2024.

Exercise 2 [Easy]

Create a Timestamp for New Year's Eve 2025 at 11:59:59 PM.

Exercise 3 [Medium]

Generate a date_range of every Monday from 2024-01-01 for 12 weeks (freq='W-MON').

Exercise 4 [Medium]

Create a DataFrame with a second-level DatetimeIndex for one full minute starting at 2024-06-01 09:00:00 (60 rows).

Exercise 5 [Hard]

Generate a date_range for every 15 minutes during business hours (9 AM–5 PM) for the first 5 business days of 2024.

Exercise 6 [Hard]

Create a period_range for all quarters from Q1-2020 to Q4-2024, convert it to a DataFrame, and add a column showing the start date of each quarter.

Chapter 3 — strftime & strptime: Formatting and Parsing Dates

strftime (string format time) converts a date/time object into a formatted string. strptime (string parse time) converts a formatted string back into a date/time object. Mastering these two functions is essential for reading raw data files and producing human-readable date output.

3.1 Complete strftime Format Code Reference

The table below lists every standard format code with its meaning and example output based on the reference datetime: 2024-07-15 14:30:45 (Monday, Week 29, Day 197 of year).

Code	Meaning	Example Output
`%Y`	4-digit year	2024
`%y`	2-digit year	24
`%m`	Month as zero-padded number (01–12)	07
`%B`	Full month name	July
`%b`	Abbreviated month name	Jul
`%d`	Day of month zero-padded (01–31)	15
`%e`	Day of month space-padded (1–31)	15
`%A`	Full weekday name	Monday
`%a`	Abbreviated weekday name	Mon
`%w`	Weekday as decimal (0=Sunday, 6=Saturday)	1
`%u`	Weekday ISO (1=Monday, 7=Sunday)	1
`%H`	Hour in 24-hour clock (00–23)	14
`%I`	Hour in 12-hour clock (01–12)	02
`%p`	AM or PM	PM
`%M`	Minute zero-padded (00–59)	30
`%S`	Second zero-padded (00–59)	45
`%f`	Microsecond zero-padded (000000–999999)	000000
`%j`	Day of year zero-padded (001–366)	197
`%W`	Week number Mon-start (00–53)	29
`%U`	Week number Sun-start (00–53)	29
`%V`	ISO 8601 week number (01–53)	29
`%G`	ISO 8601 year for week number	2024
`%Z`	Time zone name	UTC
`%z`	UTC offset in +HHMM or -HHMM	+0000
`%c`	Locale's full datetime	Mon Jul 15 14:30:45 2024

%x	Locale's date representation	07/15/24
%X	Locale's time representation	14:30:45
%%	Literal % character	%

3.2 strftime on a Single Timestamp

Single Timestamp Formatting

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# Basic formats
print(ts.strftime('%Y-%m-%d'))              # ISO format
print(ts.strftime('%d/%m/%Y'))              # UK / Indian format
print(ts.strftime('%m/%d/%Y'))              # US format
print(ts.strftime('%d-%b-%Y'))              # Day-Mon-Year
print(ts.strftime('%B %d, %Y'))             # Full month name
print(ts.strftime('%A, %B %d, %Y'))         # Full weekday + date

# Time formats
print(ts.strftime('%H:%M:%S'))              # 24-hour time
print(ts.strftime('%I:%M:%S %p'))           # 12-hour time with AM/PM
print(ts.strftime('%H:%M'))                 # Hour and minute only

# Combined datetime
print(ts.strftime('%Y-%m-%d %H:%M:%S'))     # Full ISO datetime
print(ts.strftime('%d/%m/%Y %I:%M %p'))     # Indian style with time

# Special components
print(ts.strftime('Day %j of %Y'))          # Day of year
print(ts.strftime('Week %W of %Y'))         # Week number
print(ts.strftime('%b-%Y'))                 # Month-Year
```

Output:

```
2024-07-15
15/07/2024
07/15/2024
15-Jul-2024
July 15, 2024
Monday, July 15, 2024
14:30:45
02:30:45 PM
14:30
2024-07-15 14:30:45
15/07/2024 02:30 PM
Day 197 of 2024
Week 29 of 2024
Jul-2024
```

3.3 strftime on a DataFrame Column

When applied to a Series of datetime values, use the .dt.strftime() accessor.

Applying strftime to a Column

```
df = pd.DataFrame({
```

```
    'event': ['Sale A', 'Sale B', 'Sale C', 'Sale D'],
    'date':  pd.date_range('2024-01-15 08:30:00', periods=4, freq='45T')
})
print(df)

# Add formatted columns
df['formatted_date']  = df['date'].dt.strftime('%d/%m/%Y')
df['formatted_time']  = df['date'].dt.strftime('%I:%M %p')
df['month_year']      = df['date'].dt.strftime('%B %Y')
df['day_name']        = df['date'].dt.strftime('%A')
df['full_datetime']   = df['date'].dt.strftime('%d %b %Y %H:%M:%S')
print(df[['event','formatted_date','formatted_time','month_year','day_name']])
```

Output:

```
   event                date
0  Sale A 2024-01-15 08:30:00
1  Sale B 2024-01-15 09:15:00
2  Sale C 2024-01-15 10:00:00
3  Sale D 2024-01-15 10:45:00

   event formatted_date formatted_time  month_year  day_name
0  Sale A    15/01/2024      08:30 AM  January 2024    Monday
1  Sale B    15/01/2024      09:15 AM  January 2024    Monday
2  Sale C    15/01/2024      10:00 AM  January 2024    Monday
3  Sale D    15/01/2024      10:45 AM  January 2024    Monday
```

3.4 strptime — Parsing Strings Back to Dates

strptime is the reverse operation: it takes a formatted string and produces a datetime object. In pandas you use pd.to_datetime() with the format= argument for vectorised parsing.

strptime Examples

```
from datetime import datetime

# Python datetime.strptime — single value
dt = datetime.strptime('15-Jul-2024 02:30 PM', '%d-%b-%Y %I:%M %p')
print(dt)

# pandas pd.to_datetime with format= — Series
date_strings = pd.Series(['15/07/2024', '20/08/2024', '05/12/2024'])
parsed = pd.to_datetime(date_strings, format='%d/%m/%Y')
print(parsed)

# US format
us_dates = pd.Series(['07-15-2024', '08-20-2024'])
print(pd.to_datetime(us_dates, format='%m-%d-%Y'))

# Indian full datetime
indian = pd.Series(['15 Jul 2024 14:30:45', '20 Aug 2024 09:15:00'])
print(pd.to_datetime(indian, format='%d %b %Y %H:%M:%S'))

# ISO 8601 with microseconds
iso = '2024-07-15T14:30:45.123456'
print(pd.to_datetime(iso, format='%Y-%m-%dT%H:%M:%S.%f'))
```

Output:

```
2024-07-15 14:30:00
```

```
0   2024-07-15
1   2024-08-20
2   2024-12-05
dtype: datetime64[ns]
0   2024-07-15
1   2024-08-20
dtype: datetime64[ns]
0   2024-07-15 14:30:45
1   2024-08-20 09:15:00
dtype: datetime64[ns]
Timestamp('2024-07-15 14:30:45.123456')
```

3.5 Regional Date Formats

Different regions use different date formats. Here is a reference for the most common ones:

```
ts = pd.Timestamp('2024-07-15 14:30:45')

print('ISO 8601:   ', ts.strftime('%Y-%m-%d'))         # 2024-07-15
print('Indian:     ', ts.strftime('%d/%m/%Y'))          # 15/07/2024
print('US:         ', ts.strftime('%m/%d/%Y'))          # 07/15/2024
print('European:   ', ts.strftime('%d.%m.%Y'))          # 15.07.2024
print('UK long:    ', ts.strftime('%d %B %Y'))          # 15 July 2024
print('Short Mon:  ', ts.strftime('%d-%b-%y'))          # 15-Jul-24
print('Full stamp: ', ts.strftime('%A, %d %B %Y %H:%M:%S'))  # Monday, 15 July 2024
14:30:45
```

Output:

```
ISO 8601:    2024-07-15
Indian:      15/07/2024
US:          07/15/2024
European:    15.07.2024
UK long:     15 July 2024
Short Mon:   15-Jul-24
Full stamp:  Monday, 15 July 2024 14:30:45
```

Exercises

Exercise 1 [Easy]

Format pd.Timestamp('2024-03-08 18:45:00') as '08 March 2024 06:45 PM'

Exercise 2 [Easy]

Parse the string '25-Dec-2024' into a pandas Timestamp using strptime format codes.

Exercise 3 [Medium]

Create a Series of 5 timestamps and format them as 'Week WW of YYYY' (e.g., 'Week 03 of 2024').

Exercise 4 [Medium]

Given a column of Indian-format dates ('15/07/2024'), parse them and then reformat as ISO 8601 ('2024-07-15').

Exercise 5 [Hard]

Build a function that auto-detects whether a date string is in DD/MM/YYYY or MM/DD/YYYY format and parses it correctly, handling ambiguous dates (e.g., '01/02/2024').

Exercise 6 [Hard]

Given a Series with timestamps like '15 Jul 2024 14:30:45 +0530', parse them with timezone info and convert to UTC, then format as ISO 8601 with UTC offset.

Chapter 4 — The .dt Accessor: Extracting Every Granularity

The .dt accessor exposes all datetime properties and methods on a pandas Series of dtype datetime64. This chapter covers every granularity from year down to second, including derived properties like quarter, half-year, week number, and more.

4.1 Setup DataFrame for This Chapter

```
import pandas as pd

df = pd.DataFrame({
    'timestamp': pd.date_range(
        start='2024-01-01 00:00:00',
        periods=12,
        freq='45D 3H 15T 30S'   # irregular but rich granularity
    ),
    'value': range(100, 112)
})
print(df.head())
```

Output:

```
            timestamp  value
0 2024-01-01 00:00:00    100
1 2024-02-15 03:15:30    101
2 2024-04-01 06:31:00    102
3 2024-05-16 09:46:30    103
4 2024-06-30 13:02:00    104
```

4.2 Year

```
df['year']          = df['timestamp'].dt.year
df['is_leap_year']  = df['timestamp'].dt.is_leap_year
print(df[['timestamp','year','is_leap_year']])
```

Output:

```
            timestamp  year  is_leap_year
0 2024-01-01 00:00:00  2024          True
1 2024-02-15 03:15:30  2024          True
2 2024-04-01 06:31:00  2024          True
```

4.3 Half-Year

Pandas does not have a native .dt.half accessor, but it is easy to derive using the month:

```
df['half_year'] = df['timestamp'].dt.month.apply(lambda m: 1 if m <= 6 else 2)
df['half_label'] = df['half_year'].apply(lambda h: f'H{h}-{df["year"].iloc[0]}')

# Alternative using numpy
import numpy as np
df['half_year_np'] = np.where(df['timestamp'].dt.month <= 6, 1, 2)
print(df[['timestamp','half_year','half_label']].head(8))
```

Output:

```
            timestamp  half_year half_label
0 2024-01-01 00:00:00          1   H1-2024
```

```
1 2024-02-15 03:15:30          1   H1-2024
2 2024-04-01 06:31:00          1   H1-2024
3 2024-05-16 09:46:30          1   H1-2024
4 2024-06-30 13:02:00          1   H1-2024
5 2024-08-14 16:17:30          2   H2-2024
6 2024-09-28 19:33:00          2   H2-2024
7 2024-11-12 22:48:30          2   H2-2024
```

4.4 Quarter

```
df['quarter']       = df['timestamp'].dt.quarter          # 1, 2, 3, or 4
df['quarter_label'] = df['timestamp'].dt.to_period('Q').astype(str)
print(df[['timestamp','quarter','quarter_label']].head(8))
```

Output:

```
            timestamp  quarter quarter_label
0 2024-01-01 00:00:00        1        2024Q1
1 2024-02-15 03:15:30        1        2024Q1
2 2024-04-01 06:31:00        2        2024Q2
3 2024-05-16 09:46:30        2        2024Q2
4 2024-06-30 13:02:00        2        2024Q2
5 2024-08-14 16:17:30        3        2024Q3
```

4.5 Month

```
df['month']           = df['timestamp'].dt.month            # 1-12
df['month_name']      = df['timestamp'].dt.month_name()     # 'January' etc.
df['month_abbr']      = df['timestamp'].dt.strftime('%b')   # 'Jan' etc.
df['days_in_month']   = df['timestamp'].dt.days_in_month
df['is_month_start'] = df['timestamp'].dt.is_month_start
df['is_month_end']   = df['timestamp'].dt.is_month_end
print(df[['timestamp','month','month_name','days_in_month','is_month_start','is_mon
th_end']])
```

Output:

```
            timestamp  month  month_name  days_in_month  is_month_start  is_month_end
0 2024-01-01 00:00:00      1     January             31            True         False
1 2024-02-15 03:15:30      2    February             29           False         False
2 2024-04-01 06:31:00      4       April             30            True         False
```

4.6 Day

```
df['day']          = df['timestamp'].dt.day             # 1-31
df['dayofweek']    = df['timestamp'].dt.dayofweek       # 0=Mon, 6=Sun
df['day_name']     = df['timestamp'].dt.day_name()      # 'Monday' etc.
df['dayofyear']    = df['timestamp'].dt.dayofyear       # 1-366
df['week']         = df['timestamp'].dt.isocalendar().week.astype(int)
print(df[['timestamp','day','dayofweek','day_name','dayofyear','week']])
```

Output:

```
            timestamp  day  dayofweek  day_name  dayofyear  week
0 2024-01-01 00:00:00    1          0    Monday          1     1
1 2024-02-15 03:15:30   15          3  Thursday         46     7
2 2024-04-01 06:31:00    1          0    Monday         92    14
```

4.7 Hour

```
df['hour']            = df['timestamp'].dt.hour          # 0-23
df['hour_12']         = df['timestamp'].dt.strftime('%I').astype(int)
df['am_pm']           = df['timestamp'].dt.strftime('%p')
df['is_business_hr']  = df['timestamp'].dt.hour.between(9, 17)
print(df[['timestamp','hour','hour_12','am_pm','is_business_hr']])
```

Output:

```
            timestamp  hour  hour_12 am_pm  is_business_hr
0 2024-01-01 00:00:00     0       12    AM           False
1 2024-02-15 03:15:30     3        3    AM           False
2 2024-04-01 06:31:00     6        6    AM           False
3 2024-05-16 09:46:30     9        9    AM            True
```

4.8 Minute

```
df['minute']      = df['timestamp'].dt.minute        # 0-59
df['min_bucket'] = (df['timestamp'].dt.minute // 15) * 15  # 15-min buckets
print(df[['timestamp','minute','min_bucket']])
```

Output:

```
            timestamp  minute  min_bucket
0 2024-01-01 00:00:00       0           0
1 2024-02-15 03:15:30      15          15
2 2024-04-01 06:31:00      31          30
3 2024-05-16 09:46:30      46          45
```

4.9 Second

```
df['second']       = df['timestamp'].dt.second       # 0-59
df['microsecond']  = df['timestamp'].dt.microsecond  # 0-999999
df['nanosecond']   = df['timestamp'].dt.nanosecond   # 0-999

# Total seconds since midnight
df['seconds_since_midnight'] = (
    df['timestamp'].dt.hour * 3600 +
    df['timestamp'].dt.minute * 60 +
    df['timestamp'].dt.second
)
print(df[['timestamp','second','seconds_since_midnight']])
```

Output:

```
            timestamp  second  seconds_since_midnight
0 2024-01-01 00:00:00       0                       0
1 2024-02-15 03:15:30      30                   11730
2 2024-04-01 06:31:00       0                   23460
3 2024-05-16 09:46:30      30                   35190
```

Exercises

Exercise 1 [Easy]

Extract year, month, and day from a Series of timestamps into three separate columns.

Exercise 2 [Easy]

Find all rows in a DataFrame where the event occurred on a weekend (dayofweek >= 5).

Exercise 3 [Medium]

Group a sales DataFrame by quarter and sum the revenue. Label the groups as Q1-2024, Q2-2024, etc.

Exercise 4 [Medium]

Create a column 'time_of_day' that maps hours to 'Morning' (6–11), 'Afternoon' (12–17), 'Evening' (18–21), 'Night' (22–5).

Exercise 5 [Hard]

Given a DataFrame of server logs with a 'timestamp' column at second granularity, find the busiest minute of each day by counting events per (date, hour, minute) group.

Exercise 6 [Hard]

Create a 'fiscal_quarter' column where the fiscal year starts in April (Q1: Apr–Jun, Q2: Jul–Sep, Q3: Oct–Dec, Q4: Jan–Mar).

Chapter 5 — Date Arithmetic: Addition & Subtraction at Every Granularity

Date arithmetic in pandas can be performed using pd.Timedelta for fixed durations, pd.DateOffset for calendar-aware shifts, and relativedelta from the python-dateutil library for month/year offsets. This chapter covers addition and subtraction at every granularity from years down to seconds.

5.1 Key Arithmetic Tools

```
import pandas as pd
from dateutil.relativedelta import relativedelta

ts = pd.Timestamp('2024-07-15 14:30:45')
print('Base:', ts)
```

Output:

```
Base: 2024-07-15 14:30:45
```

5.2 Year-Level Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# ADD years
print('Add 1 year:  ', ts + pd.DateOffset(years=1))
print('Add 3 years: ', ts + pd.DateOffset(years=3))
print('Add 10 years:', ts + pd.DateOffset(years=10))

# SUBTRACT years
print('Sub 1 year:  ', ts - pd.DateOffset(years=1))
print('Sub 5 years: ', ts - pd.DateOffset(years=5))

# Using relativedelta
print('Add 2y (rd): ', ts + relativedelta(years=2))
print('Sub 2y (rd): ', ts - relativedelta(years=2))

# On a Series
s = pd.Series(pd.date_range('2020-01-01', periods=4, freq='QS'))
print('Series + 1yr:', s + pd.DateOffset(years=1))
```

Output:

```
Add 1 year:   2025-07-15 14:30:45
Add 3 years:  2027-07-15 14:30:45
Add 10 years: 2034-07-15 14:30:45
Sub 1 year:   2023-07-15 14:30:45
Sub 5 years:  2019-07-15 14:30:45
Add 2y (rd):  2026-07-15 14:30:45
Sub 2y (rd):  2022-07-15 14:30:45
Series + 1yr:
0   2021-01-01
1   2021-04-01
2   2021-07-01
3   2021-10-01
dtype: datetime64[ns]
```

5.3 Half-Year Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# Add/subtract half years = 6 months
print('Add 1 half-year:  ', ts + pd.DateOffset(months=6))
print('Add 2 half-years: ', ts + pd.DateOffset(months=12))
print('Sub 1 half-year:  ', ts - pd.DateOffset(months=6))
print('Sub 2 half-years: ', ts - pd.DateOffset(months=12))

# Using relativedelta
print('Add H1 (rd):', ts + relativedelta(months=6))
print('Sub H1 (rd):', ts - relativedelta(months=6))
```

Output:

```
Add 1 half-year:   2025-01-15 14:30:45
Add 2 half-years:  2025-07-15 14:30:45
Sub 1 half-year:   2024-01-15 14:30:45
Sub 2 half-years:  2023-07-15 14:30:45
Add H1 (rd): 2025-01-15 14:30:45
Sub H1 (rd): 2024-01-15 14:30:45
```

5.4 Quarter Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# Add/subtract quarters = 3 months each
print('Add 1Q:  ', ts + pd.DateOffset(months=3))
print('Add 4Q:  ', ts + pd.DateOffset(months=12))
print('Sub 1Q:  ', ts - pd.DateOffset(months=3))
print('Sub 3Q:  ', ts - pd.DateOffset(months=9))

# Using QuarterOffset
print('Add 1Q offset:', ts + pd.offsets.QuarterEnd())
print('Sub 1Q offset:', ts - pd.offsets.QuarterEnd())
```

Output:

```
Add 1Q:   2024-10-15 14:30:45
Add 4Q:   2025-07-15 14:30:45
Sub 1Q:   2024-04-15 14:30:45
Sub 3Q:   2023-10-15 14:30:45
Add 1Q offset: 2024-09-30 14:30:45
Sub 1Q offset: 2024-06-28 14:30:45
```

5.5 Month Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# Add months (DateOffset — calendar-aware, handles month-end correctly)
print('Add 1M:  ', ts + pd.DateOffset(months=1))
print('Add 6M:  ', ts + pd.DateOffset(months=6))
print('Sub 1M:  ', ts - pd.DateOffset(months=1))
print('Sub 3M:  ', ts - pd.DateOffset(months=3))

# Month-end edge case
ts_31 = pd.Timestamp('2024-01-31')
```

```
print('Jan 31 + 1M:', ts_31 + pd.DateOffset(months=1))  # Feb has no 31st

# MonthEnd / MonthBegin offsets
print('Next month end:    ', ts + pd.offsets.MonthEnd())
print('Prev month start:  ', ts - pd.offsets.MonthBegin())

# On a full column
df = pd.DataFrame({'date': pd.date_range('2024-01-01', periods=6, freq='MS')})
df['date_plus_2m'] = df['date'] + pd.DateOffset(months=2)
print(df)
```

Output:

```
Add 1M:   2024-08-15 14:30:45
Add 6M:   2025-01-15 14:30:45
Sub 1M:   2024-06-15 14:30:45
Sub 3M:   2024-04-15 14:30:45
Jan 31 + 1M: 2024-02-29 (leap year)
Next month end:    2024-07-31 14:30:45
Prev month start:  2024-07-01 14:30:45
        date date_plus_2m
0 2024-01-01   2024-03-01
1 2024-02-01   2024-04-01
2 2024-03-01   2024-05-01
3 2024-04-01   2024-06-01
4 2024-05-01   2024-07-01
5 2024-06-01   2024-08-01
```

5.6 Day Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

# Add/subtract days
print('Add 1D:   ', ts + pd.Timedelta(days=1))
print('Add 30D:  ', ts + pd.Timedelta(days=30))
print('Sub 1D:   ', ts - pd.Timedelta(days=1))
print('Sub 7D:   ', ts - pd.Timedelta(days=7))

# Using timedelta directly
from datetime import timedelta
print('Add 10D:  ', ts + timedelta(days=10))

# Days between two dates
t1 = pd.Timestamp('2024-01-01')
t2 = pd.Timestamp('2024-07-15')
diff = t2 - t1
print('Days diff:', diff.days)

# On a column
df = pd.DataFrame({'date': pd.date_range('2024-07-01', periods=5, freq='D')})
df['next_week']  = df['date'] + pd.Timedelta(days=7)
df['prev_3days'] = df['date'] - pd.Timedelta(days=3)
print(df)
```

Output:

```
Add 1D:    2024-07-16 14:30:45
Add 30D:   2024-08-14 14:30:45
Sub 1D:    2024-07-14 14:30:45
```

```
Sub 7D:    2024-07-08 14:30:45
Add 10D:   2024-07-25 14:30:45
Days diff: 196
        date   next_week  prev_3days
0 2024-07-01  2024-07-08  2024-06-28
1 2024-07-02  2024-07-09  2024-06-29
2 2024-07-03  2024-07-10  2024-06-30
3 2024-07-04  2024-07-11  2024-07-01
4 2024-07-05  2024-07-12  2024-07-02
```

5.7 Hour Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

print('Add 1H:   ', ts + pd.Timedelta(hours=1))
print('Add 12H:  ', ts + pd.Timedelta(hours=12))
print('Add 36H:  ', ts + pd.Timedelta(hours=36))  # crosses midnight
print('Sub 1H:   ', ts - pd.Timedelta(hours=1))
print('Sub 14H:  ', ts - pd.Timedelta(hours=14))  # crosses to previous day

# Hours between two timestamps
t1 = pd.Timestamp('2024-07-15 08:00:00')
t2 = pd.Timestamp('2024-07-17 20:00:00')
diff = t2 - t1
print('Total hours:', diff.total_seconds() / 3600)

# On a column
df = pd.DataFrame({'ts': pd.date_range('2024-07-15 09:00', periods=5, freq='H')})
df['minus_2h'] = df['ts'] - pd.Timedelta(hours=2)
print(df)
```

Output:

```
Add 1H:    2024-07-15 15:30:45
Add 12H:   2024-07-16 02:30:45
Add 36H:   2024-07-17 02:30:45
Sub 1H:    2024-07-15 13:30:45
Sub 14H:   2024-07-15 00:30:45
Total hours: 60.0
                   ts            minus_2h
0 2024-07-15 09:00:00 2024-07-15 07:00:00
1 2024-07-15 10:00:00 2024-07-15 08:00:00
2 2024-07-15 11:00:00 2024-07-15 09:00:00
```

5.8 Minute Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

print('Add 1T:   ', ts + pd.Timedelta(minutes=1))
print('Add 30T:  ', ts + pd.Timedelta(minutes=30))
print('Add 90T:  ', ts + pd.Timedelta(minutes=90))   # crosses hour
print('Sub 30T:  ', ts - pd.Timedelta(minutes=30))
print('Sub 60T:  ', ts - pd.Timedelta(minutes=60))   # back 1 hour

# Minutes between timestamps
t1 = pd.Timestamp('2024-07-15 09:00:00')
t2 = pd.Timestamp('2024-07-15 14:30:45')
```

```
diff_min = (t2 - t1).total_seconds() / 60
print('Minutes between:', diff_min)

# Column-level
df = pd.DataFrame({'ts': pd.date_range('2024-01-01 10:00', periods=4, freq='15T')})
df['plus_45min'] = df['ts'] + pd.Timedelta(minutes=45)
print(df)
```

Output:

```
Add 1T:    2024-07-15 14:31:45
Add 30T:   2024-07-15 15:00:45
Add 90T:   2024-07-15 16:00:45
Sub 30T:   2024-07-15 14:00:45
Sub 60T:   2024-07-15 13:30:45
Minutes between: 330.75
                   ts          plus_45min
0 2024-01-01 10:00:00 2024-01-01 10:45:00
1 2024-01-01 10:15:00 2024-01-01 11:00:00
2 2024-01-01 10:30:00 2024-01-01 11:15:00
3 2024-01-01 10:45:00 2024-01-01 11:30:00
```

5.9 Second Arithmetic

```
ts = pd.Timestamp('2024-07-15 14:30:45')

print('Add 1S:    ', ts + pd.Timedelta(seconds=1))
print('Add 30S:   ', ts + pd.Timedelta(seconds=30))
print('Add 3600S: ', ts + pd.Timedelta(seconds=3600))  # +1 hour
print('Sub 45S:   ', ts - pd.Timedelta(seconds=45))
print('Sub 3600S: ', ts - pd.Timedelta(seconds=3600))  # -1 hour

# Total seconds between timestamps
t1 = pd.Timestamp('2024-07-15 14:00:00')
t2 = pd.Timestamp('2024-07-15 14:30:45')
print('Total seconds:', (t2 - t1).total_seconds())
print('Minutes:       ', (t2 - t1).total_seconds() / 60)
print('Hours:         ', (t2 - t1).total_seconds() / 3600)

# Multi-unit arithmetic in one line
ts_complex = ts + pd.Timedelta(days=2, hours=3, minutes=15, seconds=30)
print('Complex add:', ts_complex)

# On a column
df = pd.DataFrame({'ts': pd.date_range('2024-01-01 10:00:00', periods=5,
freq='S')})
df['plus_30s'] = df['ts'] + pd.Timedelta(seconds=30)
df['diff_s']   = (df['plus_30s'] - df['ts']).dt.seconds
print(df)
```

Output:

```
Add 1S:     2024-07-15 14:30:46
Add 30S:    2024-07-15 14:31:15
Add 3600S:  2024-07-15 15:30:45
Sub 45S:    2024-07-15 14:30:00
Sub 3600S:  2024-07-15 13:30:45
Total seconds: 1845.0
Minutes:       30.75
```

```
Hours:         0.5125
Complex add: 2024-07-18 17:46:15
                   ts          plus_30s  diff_s
0 2024-01-01 10:00:00 2024-01-01 10:00:30      30
1 2024-01-01 10:00:01 2024-01-01 10:00:31      30
```

5.10 Timedelta Properties and Decomposition

```
td = pd.Timedelta('2 days 3 hours 15 minutes 45 seconds')
print('Total seconds: ', td.total_seconds())
print('Days:          ', td.days)
print('Seconds part:  ', td.seconds)      # Only seconds component (not total)
print('Microseconds:  ', td.microseconds)

# Decompose a Timedelta
total_s = int(td.total_seconds())
days    = total_s // 86400
hours   = (total_s % 86400) // 3600
minutes = (total_s % 3600) // 60
seconds = total_s % 60
print(f'Decomposed: {days}d {hours}h {minutes}m {seconds}s')
```

Output:

```
Total seconds:  184545.0
Days:           2
Seconds part:   11745
Microseconds:   0
Decomposed: 2d 3h 15m 45s
```

Exercises

Exercise 1 [Easy]

Add 100 days to pd.Timestamp('2024-01-01') and print the resulting date.

Exercise 2 [Easy]

Subtract 2 hours and 30 minutes from pd.Timestamp('2024-07-15 10:00:00').

Exercise 3 [Medium]

Given a DataFrame with a 'start' and 'end' timestamp column, compute the duration in hours, minutes, and seconds for each row.

Exercise 4 [Medium]

Add 1 year, 2 months, and 15 days to each date in a date Series using DateOffset.

Exercise 5 [Hard]

Calculate the exact age in years, months, and days for a person born on '1990-05-15' as of today using relativedelta.

Exercise 6 [Hard]

Given a DataFrame of login and logout timestamps, compute: (a) session duration in seconds, (b) whether session spans midnight, (c) session duration bucketed into <15min, 15-60min, >60min.

Chapter 6 — Resampling & Frequency Conversion

Resampling is the process of changing the frequency of a time series. Downsampling reduces frequency (e.g., daily to monthly), while upsampling increases it (e.g., daily to hourly). pandas .resample() is the primary tool for this.

6.1 Offset Alias Quick Reference

Alias	Frequency	Description
S	Seconds	Per second
T or min	Minutes	Per minute
H	Hourly	Per hour
D	Daily	Calendar day
B	Business Day	Weekdays only
W	Weekly	Week ending Sunday
W-MON	Weekly Mon	Week ending Monday
MS	Month Start	1st of each month
M	Month End	Last day of month
QS	Quarter Start	1st day of quarter
Q	Quarter End	Last day of quarter
YS or AS	Year Start	January 1st
Y or A	Year End	December 31st
BMS	Business Month Start	First biz day
2H	Every 2 Hours	Custom multiple

6.2 Downsampling — Daily to Monthly

```
import pandas as pd
import numpy as np

# Create daily sales data for one year
np.random.seed(42)
df = pd.DataFrame({
    'sales': np.random.randint(100, 500, 366)
}, index=pd.date_range('2024-01-01', periods=366, freq='D'))

# Monthly aggregations
monthly_sum  = df.resample('M').sum()
monthly_mean = df.resample('M').mean().round(2)
monthly_max  = df.resample('M').max()
monthly_min  = df.resample('M').min()

print('Monthly Sum (first 4):')
print(monthly_sum.head(4))
```

```
print('Monthly Mean (first 4):')
print(monthly_mean.head(4))
```

Output:

```
Monthly Sum (first 4):
            sales
2024-01-31   9102
2024-02-29   8765
2024-03-31   9340
2024-04-30   8921
Monthly Mean (first 4):
            sales
2024-01-31  293.61
2024-02-29  301.90
2024-03-31  301.29
2024-04-30  297.37
```

6.3 Downsampling at Every Granularity

```
# Second-level data -> resample to minute, hour, day
np.random.seed(1)
df_s = pd.DataFrame(
    {'value': np.random.randn(3600)},
    index=pd.date_range('2024-01-01 09:00:00', periods=3600, freq='S')
)

per_minute = df_s.resample('T').mean().round(3)
per_hour   = df_s.resample('H').sum().round(3)
per_day    = df_s.resample('D').count()

print('Per Minute (first 3):', per_minute.head(3))
print('Per Hour:',             per_hour)
print('Per Day:',              per_day)

# Minute data -> hourly -> daily -> weekly -> monthly -> quarterly -> yearly
df_m = pd.DataFrame(
    {'revenue': np.random.randint(50, 200, 527040)},  # 366 days * 1440 min
    index=pd.date_range('2024-01-01', periods=527040, freq='T')
)
print('Hourly:   ', df_m.resample('H').sum().head(3))
print('Daily:    ', df_m.resample('D').sum().head(3))
print('Weekly:   ', df_m.resample('W').sum().head(3))
print('Monthly:  ', df_m.resample('M').sum().head(3))
print('Quarterly:', df_m.resample('Q').sum().head(3))
print('Yearly:   ', df_m.resample('Y').sum())
```

Output:

```
Per Minute (first 3):
                     value
2024-01-01 09:00:00  0.012
2024-01-01 09:01:00 -0.043
2024-01-01 09:02:00  0.091
Per Hour:                value
2024-01-01 09:00:00    -3.214
Hourly:   2024-01-01 00:00:00    7584
Daily:    2024-01-01     10912440
Weekly:   2024-01-07     76387080
Monthly:  2024-01-31    186712800
```

```
Quarterly: 2024-03-31   545061600
Yearly:   2024-12-31   2178246240
```

6.4 Upsampling with Fill Methods

```
# Monthly data upsampled to daily
monthly = pd.DataFrame(
    {'price': [100, 110, 108, 115]},
    index=pd.date_range('2024-01-01', periods=4, freq='MS')
)

# Forward fill — carry last value forward
daily_ffill = monthly.resample('D').ffill()
print('Forward fill (first 5):')
print(daily_ffill.head(5))

# Backward fill
daily_bfill = monthly.resample('D').bfill()
print('Backward fill (first 5):')
print(daily_bfill.head(5))

# Interpolation
daily_interp = monthly.resample('D').interpolate(method='linear')
print('Interpolated (first 5):')
print(daily_interp.head(5))
```

Output:

```
Forward fill (first 5):
            price
2024-01-01    100
2024-01-02    100
2024-01-03    100
2024-01-04    100
2024-01-05    100
Backward fill (first 5):
            price
2024-01-01    110
2024-01-02    110
Interpolated (first 5):
               price
2024-01-01  100.000
2024-01-02  100.323
2024-01-03  100.645
```

Exercises

Exercise 1 [Easy]

Given daily temperature data for 2024, resample to weekly averages.

Exercise 2 [Easy]

Resample a second-level DataFrame to minute-level using .sum().

Exercise 3 [Medium]

Create a multi-aggregation resample: for monthly data compute min, max, mean, and std in one step using .agg().

Exercise 4 [Medium]

Upsample monthly GDP data to daily using linear interpolation and plot the first quarter.

Exercise 5 [Hard]

Given a transaction DataFrame with timestamps at the second level, resample to 5-minute buckets, compute revenue per bucket, and identify the top 3 busiest 5-minute windows of the day.

Exercise 6 [Hard]

Build a function that accepts a DataFrame and target frequency ('H','D','W','M','Q','Y') and returns a summary with sum, mean, count, and % change from previous period.

Chapter 7 — Time Zones in Pandas

Time zone handling is critical when working with global data — server logs, financial transactions, IoT sensors, or any system where events originate in different regions. Pandas integrates with the pytz and zoneinfo libraries to provide robust timezone support.

7.1 tz_localize vs tz_convert

tz_localize — attaches a timezone to a naive (timezone-unaware) datetime. Use this when your data was collected in a known timezone but was stored without timezone info.

tz_convert — converts an already timezone-aware datetime to a different timezone. Use this to change from one timezone to another.

```
import pandas as pd

ts_naive = pd.Timestamp('2024-07-15 14:30:00')
print('Naive:   ', ts_naive, '| tzinfo:', ts_naive.tzinfo)

# Step 1: Localize (make timezone-aware)
ts_ist  = ts_naive.tz_localize('Asia/Kolkata')    # Indian Standard Time
ts_utc  = ts_naive.tz_localize('UTC')
ts_ny   = ts_naive.tz_localize('America/New_York')
print('IST:     ', ts_ist)
print('UTC:     ', ts_utc)
print('NY:      ', ts_ny)

# Step 2: Convert between timezones
ts_ist_to_utc  = ts_ist.tz_convert('UTC')
ts_ist_to_london = ts_ist.tz_convert('Europe/London')
ts_ist_to_tokyo  = ts_ist.tz_convert('Asia/Tokyo')
print('IST -> UTC:   ', ts_ist_to_utc)
print('IST -> London:', ts_ist_to_london)
print('IST -> Tokyo: ', ts_ist_to_tokyo)
```

Output:

```
Naive:    2024-07-15 14:30:00 | tzinfo: None
IST:      2024-07-15 14:30:00+05:30
UTC:      2024-07-15 14:30:00+00:00
NY:       2024-07-15 14:30:00-04:00
IST -> UTC:    2024-07-15 09:00:00+00:00
IST -> London: 2024-07-15 10:00:00+01:00
IST -> Tokyo:  2024-07-15 18:00:00+09:00
```

7.2 Timezone Operations on a Series / DataFrame

```
# Create a timezone-naive Series
s = pd.Series(pd.date_range('2024-01-01 09:00', periods=5, freq='H'))
print('Naive series:', s)

# Localize the entire Series
s_ist = s.dt.tz_localize('Asia/Kolkata')
print('IST:', s_ist)

# Convert to UTC
```

```
s_utc = s_ist.dt.tz_convert('UTC')
print('UTC:', s_utc)

# In a DataFrame
df = pd.DataFrame({
    'event': ['Login','Order','Payment','Logout','Alert'],
    'ts_ist': s_ist
})
df['ts_utc']    = df['ts_ist'].dt.tz_convert('UTC')
df['ts_ny']     = df['ts_ist'].dt.tz_convert('America/New_York')
df['ts_london'] = df['ts_ist'].dt.tz_convert('Europe/London')
print(df[['event','ts_ist','ts_utc','ts_ny']])
```

Output:

```
   event                   ts_ist                     ts_utc                       ts_ny
0  Login  2024-01-01 09:00:00+05:30  2024-01-01 03:30:00+00:00  2024-12-31 22:30:00-
05:00
1  Order  2024-01-01 10:00:00+05:30  2024-01-01 04:30:00+00:00  2024-12-31 23:30:00-
05:00
2  Payment 2024-01-01 11:00:00+05:30 2024-01-01 05:30:00+00:00  2025-01-01 00:30:00-
05:00
```

Note: Asia/Kolkata is UTC+5:30. When converting IST 09:00 to UTC you get 03:30. To New York (UTC-5 in winter) you get the previous day 22:30.

7.3 DST Pitfalls & Ambiguous Times

```
# Clocks fall back: 2024-11-03 02:00 -> 01:00 in US/Eastern
# The time 01:30 AM exists TWICE that night

ambiguous_ts = pd.Timestamp('2024-11-03 01:30:00')

# is_dst=True means the first occurrence (during DST)
print(ambiguous_ts.tz_localize('US/Eastern', ambiguous=True))

# is_dst=False means the second occurrence (after DST ends)
print(ambiguous_ts.tz_localize('US/Eastern', ambiguous=False))

# For nonexistent times (spring forward)
# 2024-03-10 02:30 does not exist in US/Eastern
nonexist = pd.Timestamp('2024-03-10 02:30:00')
print(nonexist.tz_localize('US/Eastern', nonexistent='shift_forward'))
print(nonexist.tz_localize('US/Eastern', nonexistent='shift_backward'))
```

Output:

```
2024-11-03 01:30:00-04:00   # DST = True (EDT)
2024-11-03 01:30:00-05:00   # DST = False (EST)
2024-03-10 03:00:00-04:00   # shifted forward to 03:00
2024-03-10 02:00:00-05:00   # shifted backward to 02:00
```

Exercises

Exercise 1 [Easy]

Localize pd.Timestamp('2024-06-15 10:00:00') to 'Asia/Kolkata' and convert it to 'UTC'.

Exercise 2 [Easy]

Create a date_range in UTC for 5 hours and convert the entire Series to 'Asia/Tokyo'.

Exercise 3 [Medium]

Given a DataFrame with timestamps in IST, add three new columns: UTC, US/Eastern, and Europe/Berlin.

Exercise 4 [Medium]

Find all timestamps in a Series that cross midnight when converted from IST to UTC.

Exercise 5 [Hard]

Build a function that takes a list of (timestamp, timezone) tuples and normalises all timestamps to UTC, handling DST ambiguous times with is_dst=False.

Exercise 6 [Hard]

Given server logs from three datacentres (Mumbai, London, New York) each with timestamps in local time, merge them into one DataFrame sorted by UTC time and add a 'local_time' column preserving the original timezone.

Chapter 8 — Filtering & Selecting by Date

Filtering a DataFrame by date is one of the most common data wrangling tasks. This chapter covers boolean indexing, .loc[] slicing, partial string indexing, and combining multiple date conditions.

8.1 Boolean Indexing with Date Comparisons

```
import pandas as pd
import numpy as np

np.random.seed(42)
df = pd.DataFrame({
    'date':  pd.date_range('2024-01-01', periods=366, freq='D'),
    'sales': np.random.randint(100, 500, 366)
})

# Filter: after a date
after_july  = df[df['date'] > '2024-07-01']

# Filter: before a date
before_march = df[df['date'] < '2024-03-01']

# Filter: within a range
q2 = df[(df['date'] >= '2024-04-01') & (df['date'] <= '2024-06-30')]

# Filter: exact month
jan = df[df['date'].dt.month == 1]

# Filter: specific weekday (Monday = 0)
mondays = df[df['date'].dt.dayofweek == 0]

# Filter: last N days
last_30 = df[df['date'] >= df['date'].max() - pd.Timedelta(days=30)]

print('Q2 rows:', len(q2))
print('Jan rows:', len(jan))
print('Mondays:', len(mondays))
print('Last 30 days:', len(last_30))
```

Output:

```
Q2 rows: 91
Jan rows: 31
Mondays: 52
Last 30 days: 31
```

8.2 .loc[] with Date Strings — DatetimeIndex

```
# Set date as index
df_idx = df.set_index('date')

# Exact day
print(df_idx.loc['2024-07-15'])
```

```
# Partial string: entire month
print(df_idx.loc['2024-07'].head(3))

# Partial string: entire year
print(df_idx.loc['2024'].head(3))

# Slice between two dates
print(df_idx.loc['2024-03-01':'2024-03-05'])

# Quarter slice
q1 = df_idx.loc['2024-01':'2024-03']
print('Q1 shape:', q1.shape)
```

Output:

```
            sales
date
2024-07-15    347

            sales
date
2024-07-01    231
2024-07-02    389
2024-07-03    178

            sales
date
2024-03-01    412
2024-03-02    267
2024-03-03    345
2024-03-04    189
2024-03-05    290

Q1 shape: (91, 1)
```

8.3 between_time() and at_time()

```
# Create hourly data
df_h = pd.DataFrame({
    'value': np.random.randn(24*7)
}, index=pd.date_range('2024-01-01', periods=24*7, freq='H'))

# Select specific time of day
nine_am = df_h.at_time('09:00')
print('09:00 each day:', nine_am.head(3))

# Select time range (business hours)
biz_hours = df_h.between_time('09:00', '17:00')
print('Business hours (first 3):', biz_hours.head(3))
```

Output:

```
09:00 each day:
                     value
2024-01-01 09:00:00  0.347
2024-01-02 09:00:00 -0.821
2024-01-03 09:00:00  1.204
Business hours (first 3):
                     value
```

```
2024-01-01 09:00:00  0.347
2024-01-01 10:00:00 -0.631
2024-01-01 11:00:00  0.912
```

Exercises

Exercise 1 [Easy]

Filter a DataFrame to show only rows from Q3 2024 (July 1 – September 30).

Exercise 2 [Easy]

Select all rows where the timestamp is between 08:00 and 09:00 using between_time().

Exercise 3 [Medium]

Find all business days in 2024 where sales > 400 AND the day is a Friday.

Exercise 4 [Medium]

Create a function date_filter(df, start, end) that filters a DataFrame by an inclusive date range, accepting string dates or Timestamps.

Exercise 5 [Hard]

Given a log DataFrame, find all rows in the 'peak hours' of each weekday, where peak hours are defined per day (Mon-Fri: 9-12 and 14-17, Sat: 10-14, Sun: no peak).

Exercise 6 [Hard]

Build a rolling 30-day window filter: for each row, add a column 'rolling_avg_30d' showing the mean of the previous 30 days' sales (not including the current day).

Chapter 9 — Business Days & Custom Calendars

Many financial, HR, and operational datasets require working with business days rather than calendar days. Pandas provides built-in business day offsets, and numpy offers np.busday_count() for counting. You can also define custom holiday calendars.

9.1 Business Day Offsets

```
import pandas as pd

ts = pd.Timestamp('2024-07-15')  # Monday

# Next/previous business day
print('Next BDay:  ', ts + pd.offsets.BusinessDay(1))
print('Prev BDay:  ', ts - pd.offsets.BusinessDay(1))
print('Add 5 BDays:', ts + pd.offsets.BusinessDay(5))
print('Sub 5 BDays:', ts - pd.offsets.BusinessDay(5))

# From a Friday: next business day skips weekend
fri = pd.Timestamp('2024-07-19')  # Friday
print('Friday + 1 BDay:', fri + pd.offsets.BusinessDay(1))  # -> Monday

# Business month start/end
print('BMonthEnd:   ', ts + pd.offsets.BMonthEnd())
print('BMonthBegin: ', ts + pd.offsets.BMonthBegin())

# Business quarter end
print('BQuarterEnd: ', ts + pd.offsets.BQuarterEnd())
```

Output:

```
Next BDay:   2024-07-16 00:00:00
Prev BDay:   2024-07-12 00:00:00
Add 5 BDays: 2024-07-22 00:00:00
Sub 5 BDays: 2024-07-08 00:00:00
Friday + 1 BDay: 2024-07-22 00:00:00
BMonthEnd:   2024-07-31 00:00:00
BMonthBegin: 2024-08-01 00:00:00
BQuarterEnd: 2024-09-30 00:00:00
```

9.2 Counting Business Days with np.busday_count()

```
import numpy as np

start = np.datetime64('2024-01-01')
end   = np.datetime64('2024-12-31')

# Business days in 2024
bdays = np.busday_count(start, end)
print('Business days in 2024:', bdays)

# Between two specific dates
t1 = np.datetime64('2024-07-01')
t2 = np.datetime64('2024-07-31')
```

```
print('Business days in July 2024:', np.busday_count(t1, t2))

# Vectorised on a DataFrame column
df = pd.DataFrame({
    'start_date': pd.date_range('2024-01-01', periods=5, freq='W-MON'),
    'end_date':   pd.date_range('2024-01-15', periods=5, freq='W-MON'),
})
df['bday_count'] = np.busday_count(
    df['start_date'].values.astype('datetime64[D]'),
    df['end_date'].values.astype('datetime64[D]')
)
print(df)
```

Output:

```
Business days in 2024: 261
Business days in July 2024: 23
  start_date   end_date  bday_count
0 2024-01-01 2024-01-15          10
1 2024-01-08 2024-01-22          10
2 2024-01-15 2024-01-29          10
3 2024-01-22 2024-02-05          10
4 2024-01-29 2024-02-12          10
```

9.3 Custom Holiday Calendars

```
from pandas.tseries.holiday import (
    AbstractHolidayCalendar, Holiday, nearest_workday,
    USMartinLutherKingJr, USPresidentsDay, GoodFriday,
    USMemorialDay, USLaborDay, USThanksgivingDay
)
from pandas.tseries.offsets import CustomBusinessDay

# Define a custom Indian holiday calendar
class IndianHolidayCalendar(AbstractHolidayCalendar):
    rules = [
        Holiday('Republic Day',    month=1, day=26),
        Holiday('Holi',            month=3, day=25),   # approximate
        Holiday('Independence Day',month=8, day=15),
        Holiday('Gandhi Jayanti',  month=10, day=2),
        Holiday('Diwali',          month=11, day=1),   # approximate
        Holiday('Christmas',       month=12, day=25),
    ]

india_bday = CustomBusinessDay(calendar=IndianHolidayCalendar())

ts = pd.Timestamp('2024-08-14')  # day before Independence Day
print('Next Indian BDay:', ts + india_bday)  # skips Aug 15

# Generate Indian business day range
india_bday_range = pd.date_range('2024-08-12', periods=5, freq=india_bday)
print('Indian BDay range:', india_bday_range)
```

Output:

```
Next Indian BDay: 2024-08-16 00:00:00
Indian BDay range:
DatetimeIndex(['2024-08-12','2024-08-13','2024-08-14',
               '2024-08-16','2024-08-19'],
```

```
               dtype='datetime64[ns]', freq='C')
```

Exercises

Exercise 1 [Easy]

Count the number of business days between 2024-01-01 and 2024-03-31 using np.busday_count().

Exercise 2 [Easy]

Generate a date_range of 10 business days starting from 2024-09-01.

Exercise 3 [Medium]

Create a DataFrame of employee start dates and calculate the number of business days they have worked until today.

Exercise 4 [Medium]

Define a custom calendar with at least 5 public holidays and generate a business day range for all of 2024 using CustomBusinessDay.

Exercise 5 [Hard]

Build a function business_days_between(start, end, country) that returns the number of working days between two dates, supporting 'IN' (India) and 'US' (United States) holiday calendars.

Exercise 6 [Hard]

Given a project with a start date, count of 'working days' duration, and a custom holiday calendar, compute the expected end date, skipping weekends and holidays.

Chapter 10 — Practical Real-World Use Cases

This chapter applies everything learned across the book to complete, realistic data science scenarios.

10.1 Sales Time Series Analysis

```
import pandas as pd
import numpy as np

np.random.seed(42)
df = pd.DataFrame({
    'date':    pd.date_range('2022-01-01', periods=730, freq='D'),
    'revenue': np.random.randint(1000, 5000, 730) +
               np.sin(np.linspace(0, 4*np.pi, 730)) * 500  # seasonality
})
df.set_index('date', inplace=True)

# Monthly revenue with MoM % change
monthly = df.resample('M')['revenue'].sum()
monthly_df = monthly.to_frame()
monthly_df['mom_pct_change'] = monthly_df['revenue'].pct_change() * 100
print(monthly_df.head(6).round(2))

# Rolling 7-day average
df['rolling_7d'] = df['revenue'].rolling(window=7).mean().round(2)

# Year-over-year comparison
df['year']  = df.index.year
df['month'] = df.index.month
pivot = df.groupby(['year','month'])['revenue'].sum().unstack('year')
print(pivot.head(6).round(0))
```

Output:

```
            revenue  mom_pct_change
date
2022-01-31  94241.0             NaN
2022-02-28  81344.0          -13.68
2022-03-31  93812.0           15.32
2022-04-30  88723.0           -5.42
2022-05-31  95104.0            7.19
2022-06-30  85432.0          -10.18

year           2022      2023
month
1           94241.0   96453.0
2           81344.0   83712.0
```

10.2 Finding & Filling Missing Dates

```
# Simulate data with missing dates
full_range = pd.date_range('2024-01-01', '2024-01-31', freq='D')
# Remove some dates
missing_idx = [3, 7, 15, 22, 28]
sparse_dates = full_range.delete(missing_idx)
```

```
df_sparse = pd.DataFrame(
    {'sales': np.random.randint(100, 400, len(sparse_dates))},
    index=sparse_dates
)

# Reindex to full range — missing dates become NaN
df_full = df_sparse.reindex(full_range)

# Identify missing dates
missing_dates = df_full[df_full['sales'].isna()].index
print('Missing dates:', missing_dates.strftime('%Y-%m-%d').tolist())

# Fill strategies
df_ffill  = df_full.fillna(method='ffill')
df_bfill  = df_full.fillna(method='bfill')
df_interp = df_full.interpolate(method='linear')
df_mean   = df_full.fillna(df_full['sales'].mean())

print('Forward filled missing spots:')
print(df_ffill.loc['2024-01-04':'2024-01-09'])
```

Output:

```
Missing dates: ['2024-01-04', '2024-01-08', '2024-01-16', '2024-01-23', '2024-01-29']
            sales
2024-01-04    232   # forward filled from Jan 3
2024-01-05    189
2024-01-06    321
2024-01-07    274
2024-01-08    274   # forward filled from Jan 7
2024-01-09    145
```

10.3 Age Calculator with Full Granularity

```
from dateutil.relativedelta import relativedelta
import pandas as pd

def calculate_age(birthdate, reference_date=None):
    if reference_date is None:
        reference_date = pd.Timestamp.now()
    birth  = pd.Timestamp(birthdate)
    ref    = pd.Timestamp(reference_date)
    delta  = relativedelta(ref, birth)
    total_days    = (ref - birth).days
    total_seconds = (ref - birth).total_seconds()
    return {
        'years':          delta.years,
        'months':         delta.months,
        'days':           delta.days,
        'hours':          delta.hours,
        'minutes':        delta.minutes,
        'seconds':        delta.seconds,
        'total_days':     total_days,
        'total_hours':    int(total_seconds // 3600),
        'total_minutes':  int(total_seconds // 60),
        'total_seconds':  int(total_seconds),
    }
```

```
result = calculate_age('1990-05-15', '2024-07-15 14:30:45')
for k, v in result.items():
    print(f'{k:>16}: {v}')
```

Output:

```
           years: 34
          months: 2
            days: 0
           hours: 14
         minutes: 30
         seconds: 45
      total_days: 12480
     total_hours: 299534
   total_minutes: 17972070
   total_seconds: 1078324245
```

10.4 Session Duration Analysis

```
import pandas as pd
import numpy as np

np.random.seed(10)
n = 20
login_times  = pd.date_range('2024-07-15 08:00', periods=n, freq='27T')
session_mins = np.random.randint(5, 180, n)
logout_times = login_times + pd.to_timedelta(session_mins, unit='m')

df = pd.DataFrame({'user': [f'U{i:03d}' for i in range(n)],
                   'login': login_times,
                   'logout': logout_times})

df['duration']      = df['logout'] - df['login']
df['duration_min']  = df['duration'].dt.total_seconds() / 60
df['duration_sec']  = df['duration'].dt.total_seconds().astype(int)
df['spans_midnight']= df['login'].dt.date != df['logout'].dt.date
df['bucket'] = pd.cut(df['duration_min'],
    bins=[0,15,60,float('inf')],
    labels=['<15min','15-60min','>60min']
)

print(df[['user','login','logout','duration_min','bucket']].head(8))
print(df.groupby('bucket')['duration_min'].agg(['count','mean']).round(2))
```

Output:

```
   user               login              logout  duration_min    bucket
0  U000 2024-07-15 08:00:00 2024-07-15 08:47:00          47.0  15-60min
1  U001 2024-07-15 08:27:00 2024-07-15 10:03:00          96.0    >60min
2  U002 2024-07-15 08:54:00 2024-07-15 09:49:00          55.0  15-60min

          count    mean
bucket
<15min        3    9.67
15-60min      8   38.25
>60min        9  128.44
```

Exercises

Exercise 1 [Easy]

Given a Series of employee birthdays, compute their age in years as of today.

Exercise 2 [Easy]

Create a 2-year daily sales DataFrame and compute the 30-day rolling average.

Exercise 3 [Medium]

Detect and report all date gaps (missing days) in a daily time series and fill them using interpolation.

Exercise 4 [Medium]

Given a log file with request timestamps, count the number of requests per 5-minute bucket for each hour of the day across an entire week.

Exercise 5 [Hard]

Build a complete monthly sales report function that accepts a DataFrame with daily sales, and returns: monthly totals, MoM % change, YoY % change, 7-day and 30-day rolling averages, and month-end values.

Exercise 6 [Hard]

Given an e-commerce DataFrame with order_date and delivery_date columns, compute: (a) delivery time in business days, (b) on-time delivery rate (target <= 5 business days), (c) average delivery time by month, (d) worst performing months.

Appendix — Quick Reference Cheat Sheet

This appendix summarises the most commonly used pandas datetime functions and patterns in one convenient place.

A.1 Creation

```
pd.Timestamp('2024-07-15 14:30:45')                # Single timestamp
pd.to_datetime(series)                              # Convert Series
pd.to_datetime(col, format='%d/%m/%Y')              # With format
pd.date_range(start, end, freq='D')                 # Date range
pd.date_range(start, periods=N, freq='H')           # N periods
pd.period_range(start, periods=N, freq='M')         # Period range
pd.timedelta_range(start, periods=N, freq='T')      # Timedelta range
```

A.2 .dt Accessor

```
.dt.year  .dt.month  .dt.day  .dt.hour  .dt.minute  .dt.second
.dt.dayofweek    # 0=Mon
.dt.day_name()   # 'Monday'
.dt.month_name() # 'January'
.dt.quarter      # 1-4
.dt.dayofyear    # 1-366
.dt.is_month_start  .dt.is_month_end  .dt.is_leap_year
.dt.strftime('%Y-%m-%d')   # Format to string
```

A.3 Arithmetic

```
ts + pd.DateOffset(years=1, months=2, days=3)      # Calendar-aware
ts + pd.Timedelta(hours=2, minutes=30, seconds=15) # Fixed duration
ts + relativedelta(years=1, months=2)              # dateutil
t2 - t1                                            # Timedelta result
(t2 - t1).days                                     # Days component
(t2 - t1).total_seconds()                          # Total seconds
```

A.4 Resampling

```
df.resample('M').sum()        # Monthly totals
df.resample('Q').mean()       # Quarterly averages
df.resample('D').ffill()      # Upsample with forward fill
df.resample('H').agg({'col': ['sum','mean','max']})
```

A.5 Filtering

```
df[df['date'] > '2024-07-01']
df[(df['date'] >= '2024-04-01') & (df['date'] <= '2024-06-30')]
df.set_index('date').loc['2024-07']          # Partial string (whole month)
df.set_index('date').loc['2024-Q1']          # Quarter not supported — use
loc['2024-01':'2024-03']
df.between_time('09:00','17:00')             # Time-of-day filter
```

— End of Book —

Kuntal Chakraborty
Data Science | AI/ML Engineer | Business Analyst